
Session 10 (AIML) - Assignment Questions

Name: **Vaishnavi Praveen Rathi**

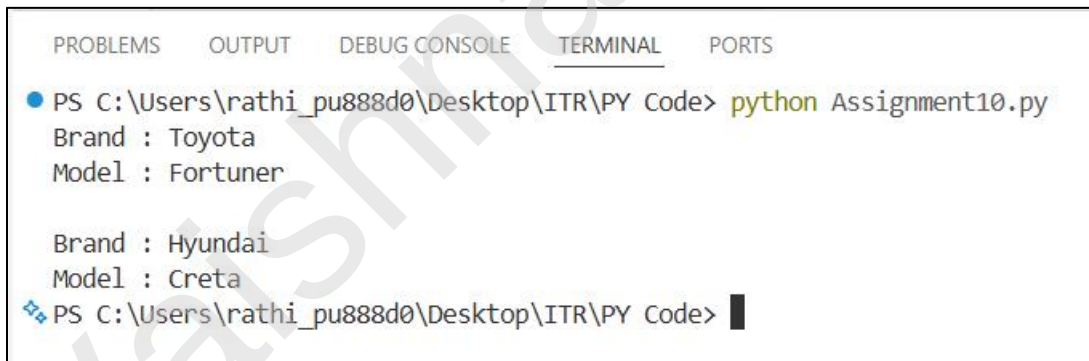
College: **Zeal Polytechnic, Narhe, Pune**

Branch: **Artificial Intelligence and Machine Learning (AIML)**

Domain: **AIML (Artificial Intelligence and Machine Learning)**

Q1. Basic Class and Object

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment10.py
Brand : Toyota
Model : Fortuner

Brand : Hyundai
Model : Creta
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 10 """
```

```
'''
```

Q1. (Basic Class and Object)

Create a class named Car with:

Attributes: brand and model (set using __init__).

A method display_info() that prints the brand and model.

Create two objects of the Car class with different values and call the method for both.

```
'''
```

```
# Create a class named Car
```

```
class Car:
```

```
    # Constructor to initialize brand and model
```

```
    def __init__(self, brand, model):
```

```
        self.brand = brand
```

```
        self.model = model
```

```
    # Method to display car information
```

```
    def display_info(self):
```

```
        print("Brand :", self.brand)
```

```
        print("Model :", self.model)
```

```
# Create first object
```

```
car1 = Car("Toyota", "Fortuner")
```

```
# Create second object
```

```
car2 = Car("Hyundai", "Creta")
```

```
# Call method for first object
```

```
car1.display_info()
```

```
print()
```

```
# Call method for second object
```

```
car2.display_info()
```

```
'''Explanation in Comments'''
```

```
# Class Car is created using the class keyword.
```

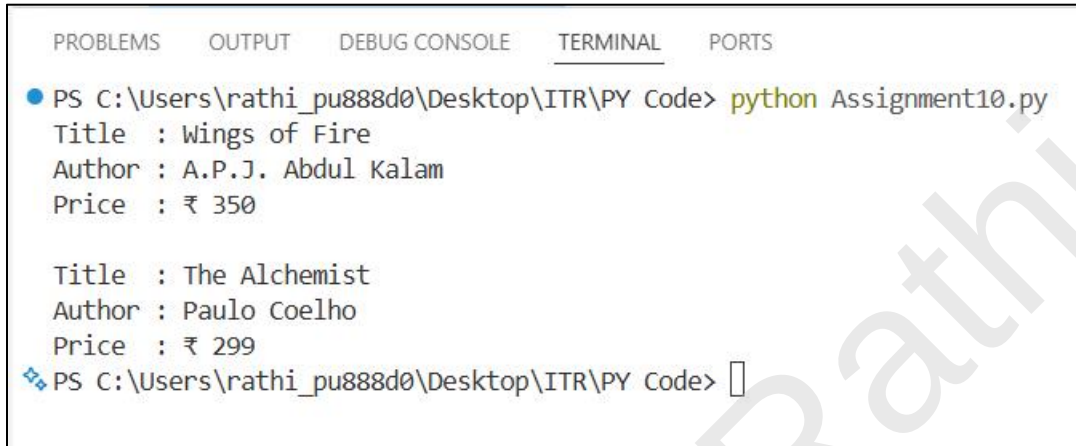
```
# The __init__() constructor initializes brand and model attributes.
```

```
# self.brand stores the brand name of the car.  
# self.model stores the model name of the car.  
# display_info() method prints the brand and model details.  
# Two objects (car1 and car2) are created with different values.  
# The display_info() method is called for both objects to display their information.
```

Vaishnavi Rathni

Q2. Using init Constructor

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment10.py
Title : Wings of Fire
Author : A.P.J. Abdul Kalam
Price : ₹ 350

Title : The Alchemist
Author : Paulo Coelho
Price : ₹ 299
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

""" ASSIGNMENT 10 """

'''

Q2. (Using __init__ Constructor)

Write a class Book that takes title, author, and price as parameters in __init__.

Create a method show_details() to print all the book information.

Create at least two book objects and display their details.

'''

Create a class named Book

class Book:

Constructor to initialize title, author and price

def __init__(self, title, author, price):

self.title = title

self.author = author

self.price = price

Method to display book details

def show_details(self):

```
print("Title :", self.title)
print("Author :", self.author)
print("Price : ₹", self.price)

# Create first book object
book1 = Book("Wings of Fire", "A.P.J. Abdul Kalam", 350)

# Create second book object
book2 = Book("The Alchemist", "Paulo Coelho", 299)

# Display details of first book
book1.show_details()

print()

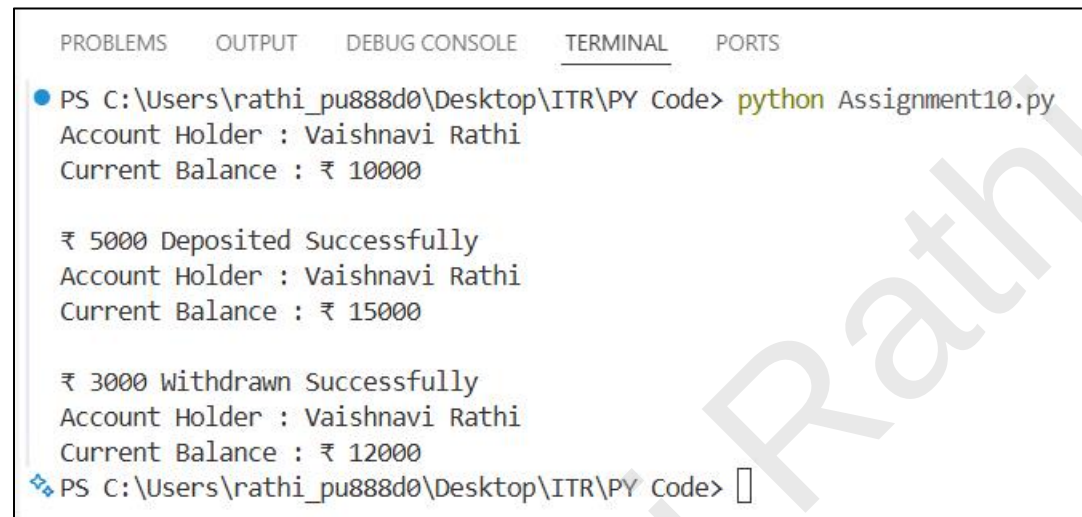
# Display details of second book
book2.show_details()

'''Explanation in Comments'''

# Class Book is created using the class keyword.
# The __init__() constructor initializes title, author and price.
# self.title stores the book title.
# self.author stores the author name.
# self.price stores the price of the book.
# show_details() method prints all book information.
# Two book objects (book1 and book2) are created.
# The show_details() method is called for both objects.
```

Q3. Instance Methods and self

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment10.py
Account Holder : Vaishnavi Rathi
Current Balance : ₹ 10000

₹ 5000 Deposited Successfully
Account Holder : Vaishnavi Rathi
Current Balance : ₹ 15000

₹ 3000 Withdrawn Successfully
Account Holder : Vaishnavi Rathi
Current Balance : ₹ 12000
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

""" ASSIGNMENT 10 """

'''

Q3. (Instance Methods and self)

Create a class BankAccount with:

Attributes:

account_holder and balance (initialized in __init__).

Methods:

deposit(amount) — add amount to balance.

withdraw(amount) — subtract amount from balance
(only if sufficient balance).

show_balance() — print current balance.

Create an object and demonstrate all methods.

'''

Create a class named BankAccount

class BankAccount:

Constructor to initialize account holder and balance

def __init__(self, account_holder, balance):

```

self.account_holder = account_holder
self.balance = balance

# Method to deposit money
def deposit(self, amount):
    self.balance += amount
    print("₹", amount, "Deposited Successfully")

# Method to withdraw money
def withdraw(self, amount):
    if amount <= self.balance:
        self.balance -= amount
        print("₹", amount, "Withdrawn Successfully")
    else:
        print("Insufficient Balance")

# Method to display current balance
def show_balance(self):
    print("Account Holder :", self.account_holder)
    print("Current Balance : ₹", self.balance)

# Create an object
account1 = BankAccount("Vaishnavi Rath", 10000)

# Show initial balance
account1.show_balance()

print()

# Deposit money
account1.deposit(5000)

# Show updated balance
account1.show_balance()

print()

# Withdraw money
account1.withdraw(3000)

# Show updated balance
account1.show_balance()

```

'''Explanation in Comments'''

```

# Class BankAccount is created using the class keyword.
# The __init__() constructor initializes account_holder and balance.
# deposit() method adds the given amount to balance.

```

```
# withdraw() method deducts amount if sufficient balance exists.  
# show_balance() method displays account holder and current balance.  
# account1 object is created with initial balance of ₹10000.  
# Deposit and withdrawal operations are demonstrated.
```

Vaishnavi Rathni

Q4.Method with Parameters

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment10.py
Name : Lavanya Tribhuvan
Marks : 75
Grade : Pass

Name : Ananya Kulkarni
Marks : 38
Grade : Fail

Name : Swanandi Shimpi
Marks : 55
Grade : Pass
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

""" ASSIGNMENT 10 """

'''

Q4. (Method with Parameters)

Create a class Student with attributes name and marks.

Add a method calculate_grade() that returns "Pass" if marks $\geq$ 40, otherwise "Fail".

Create 3 student objects, display their details, and print their grades.

'''

Create a class named Student
class Student:

Constructor to initialize name and marks
def __init__(self, name, marks):
 self.name = name
 self.marks = marks

Method to calculate grade

```

def calculate_grade(self):
    if self.marks >= 40:
        return "Pass"
    else:
        return "Fail"

# Method to display student details
def display_details(self):
    print("Name :", self.name)
    print("Marks :", self.marks)
    print("Grade :", self.calculate_grade())

# Create 3 student objects
student1 = Student("Lavanya Tribhuvan", 75)
student2 = Student("Ananya Kulkarni", 38)
student3 = Student("Swanandi Shimpi", 55)

# Display details of all students
student1.display_details()

print()

student2.display_details()

print()

student3.display_details()

'''Explanation in Comments'''

# Class Student is created using the class keyword.
# The __init__() constructor initializes name and marks.
# calculate_grade() checks whether marks are greater than
# or equal to 40 and returns Pass or Fail.
# display_details() prints student information and grade.
# Three student objects are created with different marks.
# The details and grades of all students are displayed.

```

Q5. Multiple Methods

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment10.py
Employee Name : Vaishnavi Rathi
Salary       : ₹ 50000

Salary Increased by ₹ 5000
New Salary   : ₹ 55000

Yearly Bonus : ₹ 5500.0
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 10 """
```

```
'''
```

Q5. (Multiple Methods)

Create a class Employee with:

Attributes:

name, salary (use `__init__`).

Methods:

`display_details()`

`give_raise(amount)` — increase salary and print new salary.

`yearly_bonus()` — return 10% of current salary.

Create an employee object and demonstrate all methods.

```
'''
```

```
# Create a class named Employee
```

```
class Employee:
```

```
    # Constructor to initialize name and salary
```

```
    def __init__(self, name, salary):
```

```
        self.name = name
```

```
        self.salary = salary
```

```

# Method to display employee details
def display_details(self):
    print("Employee Name :", self.name)
    print("Salary      : ₹", self.salary)

# Method to increase salary
def give_raise(self, amount):
    self.salary += amount
    print("Salary Increased by ₹", amount)
    print("New Salary : ₹", self.salary)

# Method to calculate yearly bonus
def yearly_bonus(self):
    return self.salary * 0.10

# Create an employee object
employee1 = Employee("Vaishnavi Rath", 50000)

# Display employee details
employee1.display_details()

print()

# Give salary raise
employee1.give_raise(5000)

print()

# Display yearly bonus
print("Yearly Bonus : ₹", employee1.yearly_bonus())

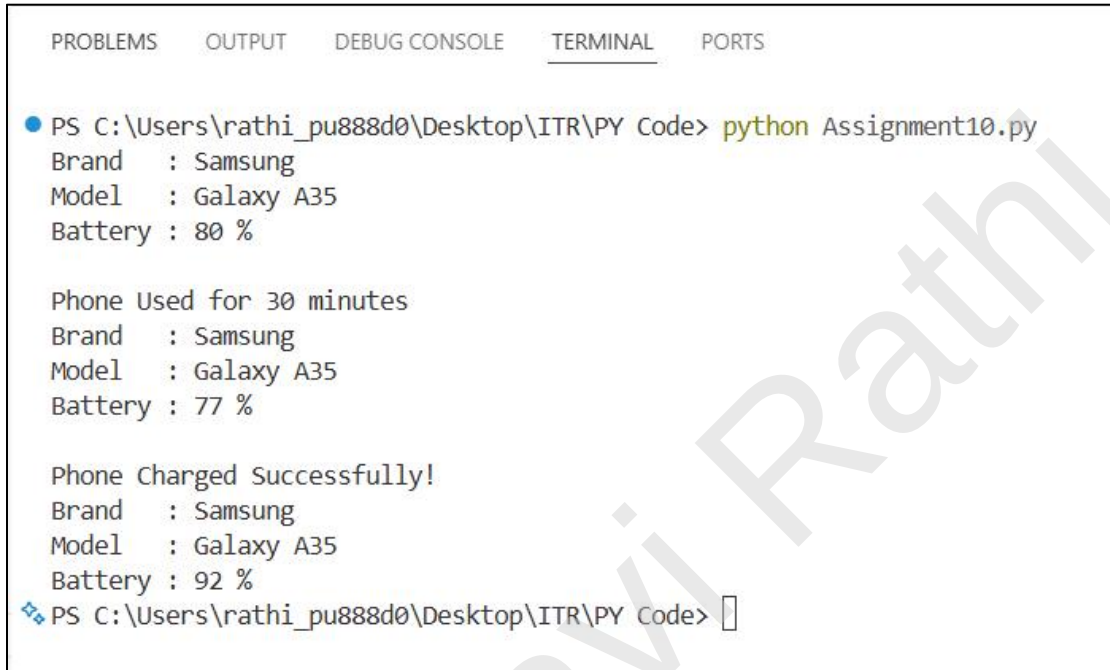
"""Explanation in Comments"""

# Class Employee is created using the class keyword.
# The __init__() constructor initializes name and salary.
# display_details() prints employee information.
# give_raise() increases the salary by the given amount.
# yearly_bonus() returns 10% of the current salary.
# An employee object is created and all methods are demonstrated.

```

Q6. Real World Object Modelling

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

• PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment10.py
Brand   : Samsung
Model   : Galaxy A35
Battery : 80 %

Phone Used for 30 minutes
Brand   : Samsung
Model   : Galaxy A35
Battery : 77 %

Phone Charged Successfully!
Brand   : Samsung
Model   : Galaxy A35
Battery : 92 %
❖ PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 10 """
```

```
'''
```

Q6. (Real-world Object Modeling)

Create a class `MobilePhone` with attributes:
`brand`, `model`, `battery_percentage`.

Add these methods:

`charge(percent)` — increase battery.

`use_phone(minutes)` — reduce battery by 1% per 10 minutes.

`show_status()` — print current status.

Create an object and simulate some usage.

```
'''
```

```
# Create a class named MobilePhone
```

```
class MobilePhone:
```

```
    # Constructor to initialize mobile details
```

```
    def __init__(self, brand, model, battery_percentage):
```

```

self.brand = brand
self.model = model
self.battery_percentage = battery_percentage

# Method to charge the phone
def charge(self, percent):
    self.battery_percentage += percent

    if self.battery_percentage > 100:
        self.battery_percentage = 100

    print("Phone Charged Successfully!")

# Method to use the phone
def use_phone(self, minutes):
    battery_used = minutes // 10
    self.battery_percentage -= battery_used

    if self.battery_percentage < 0:
        self.battery_percentage = 0

    print("Phone Used for", minutes, "minutes")

# Method to display phone status
def show_status(self):
    print("Brand :", self.brand)
    print("Model :", self.model)
    print("Battery :", self.battery_percentage, "%")

# Create a MobilePhone object
phone1 = MobilePhone("Samsung", "Galaxy A35", 80)

# Display initial status
phone1.show_status()

print()

# Simulate phone usage
phone1.use_phone(30)

# Display updated status
phone1.show_status()

print()

# Charge the phone
phone1.charge(15)

# Display final status

```

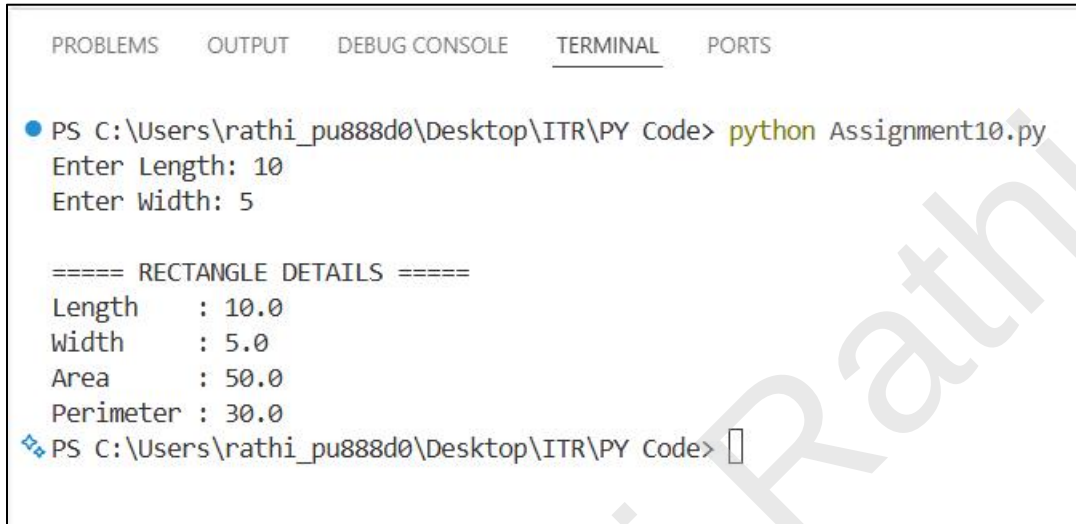
```
phone1.show_status()
```

"Explanation in Comments"

```
# Class MobilePhone is created using the class keyword.  
# The __init__() constructor initializes brand, model,  
# and battery percentage.  
# charge() increases the battery percentage.  
# use_phone() decreases battery by 1% for every 10 minutes.  
# show_status() displays the current phone details.  
# A MobilePhone object is created and usage is simulated.
```

Q7. Combining Concepts

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\rathi_pu88d0\Desktop\ITR\PY Code> python Assignment10.py
Enter Length: 10
Enter Width: 5

===== RECTANGLE DETAILS =====
Length      : 10.0
Width       : 5.0
Area        : 50.0
Perimeter   : 30.0
PS C:\Users\rathi_pu88d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 10 """
```

```
'''
```

Q7. (Combining Concepts)

Create a class Rectangle with attributes length and width.

Add methods:

area() — return the area.

perimeter() — return the perimeter.

display() — print length, width, area, and perimeter.

Take input from user for length and width,
create object, and display results.

```
'''
```

```
# Create a class named Rectangle
```

```
class Rectangle:
```

```
    # Constructor to initialize length and width
```

```
    def __init__(self, length, width):
```

```
        self.length = length
```

```
        self.width = width
```

```
    # Method to calculate area
```

```

def area(self):
    return self.length * self.width

# Method to calculate perimeter
def perimeter(self):
    return 2 * (self.length + self.width)

# Method to display rectangle details
def display(self):
    print("\n===== RECTANGLE DETAILS =====")
    print("Length   :", self.length)
    print("Width    :", self.width)
    print("Area     :", self.area())
    print("Perimeter :", self.perimeter())

# Take input from user
length = float(input("Enter Length: "))
width = float(input("Enter Width: "))

# Create Rectangle object
rectangle1 = Rectangle(length, width)

# Display details
rectangle1.display()

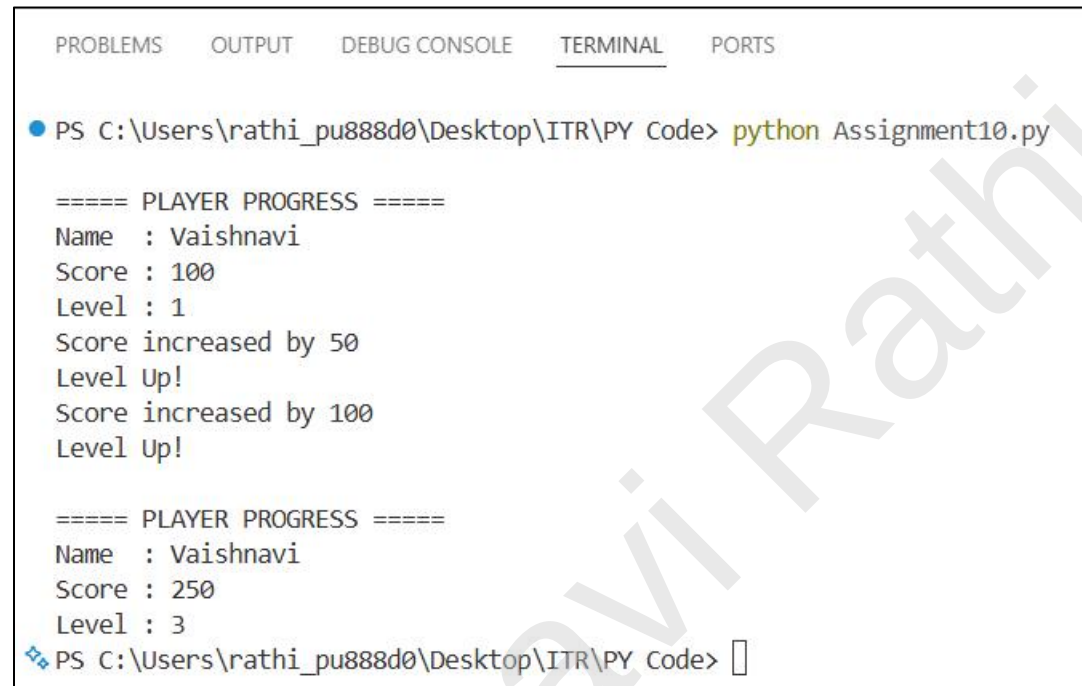
"Explanation in Comments"

# Class Rectangle is created using the class keyword.
# The __init__() constructor initializes length and width.
# area() returns the area of the rectangle.
# perimeter() returns the perimeter of the rectangle.
# display() prints all rectangle details.
# User enters length and width values.
# An object is created and the results are displayed.

```

Q8. Updating Attributes

Output Screenshot:



```
PS C:\Users\rathi_pu88d0\Desktop\ITR\PY Code> python Assignment10.py

===== PLAYER PROGRESS =====
Name : Vaishnavi
Score : 100
Level : 1
Score increased by 50
Level Up!
Score increased by 100
Level Up!

===== PLAYER PROGRESS =====
Name : Vaishnavi
Score : 250
Level : 3
PS C:\Users\rathi_pu88d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 10 """
```

```
'''
```

Q8. (Updating Attributes)

Create a class Player with attributes name, score, and level.

Add methods:

increase_score(points)

level_up() — increase level by 1.

show_progress()

Create an object, update score and level multiple times,
and show progress.

```
'''
```

```
# Create a class named Player
```

```
class Player:
```

```

# Constructor to initialize player details
def __init__(self, name, score, level):
    self.name = name
    self.score = score
    self.level = level

# Method to increase score
def increase_score(self, points):
    self.score += points
    print(f"Score increased by {points}")

# Method to increase level
def level_up(self):
    self.level += 1
    print("Level Up!")

# Method to display player progress
def show_progress(self):
    print("\n===== PLAYER PROGRESS =====")
    print("Name :", self.name)
    print("Score :", self.score)
    print("Level :", self.level)

# Create Player object
player1 = Player("Vaishnavi", 100, 1)

# Show initial progress
player1.show_progress()

# Update score and level multiple times
player1.increase_score(50)
player1.level_up()

player1.increase_score(100)
player1.level_up()

# Show updated progress
player1.show_progress()

```

"Explanation in Comments"

```
# Class Player is created using the class keyword.  
# The __init__() constructor initializes name, score, and level.  
# increase_score() adds points to the player's score.  
# level_up() increases the player's level by 1.  
# show_progress() displays the current player details.  
# An object is created and updated multiple times.  
# Final progress is displayed after all updates.
```

Q9.Debugging OOP Code

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu88d0\Desktop\ITR\PY Code> python Assignment10.py
My name is Rahul and I am 25 years old.
PS C:\Users\rathi_pu88d0\Desktop\ITR\PY Code> 
```

Program Code:

""" ASSIGNMENT 10 """

'''

Q9. (Debugging OOP Code)

The following code has errors. Correct it and add proper comments explaining the fixes.

'''

Create a class named Person

class Person:

Constructor to initialize name and age

def __init__(self, name, age):

Fixed: Used self.name and self.age to store values

self.name = name

self.age = age

Method to introduce the person

def introduce(self):

Fixed: Used f-string for proper output formatting

print(f'My name is {self.name} and I am {self.age} years old.')

Create an object of Person class

p1 = Person("Rahul", 25)

Call the introduce method

p1.introduce()

"Explanation in Comments"

Error 1:

__init__() was missing the self parameter.

Fixed: def __init__(self, name, age)

Error 2:

Variables were assigned incorrectly.

Fixed: self.name = name and self.age = age

Error 3:

introduce() method was missing self parameter.

Fixed: def introduce(self)

Error 4:

print statement syntax was incorrect.

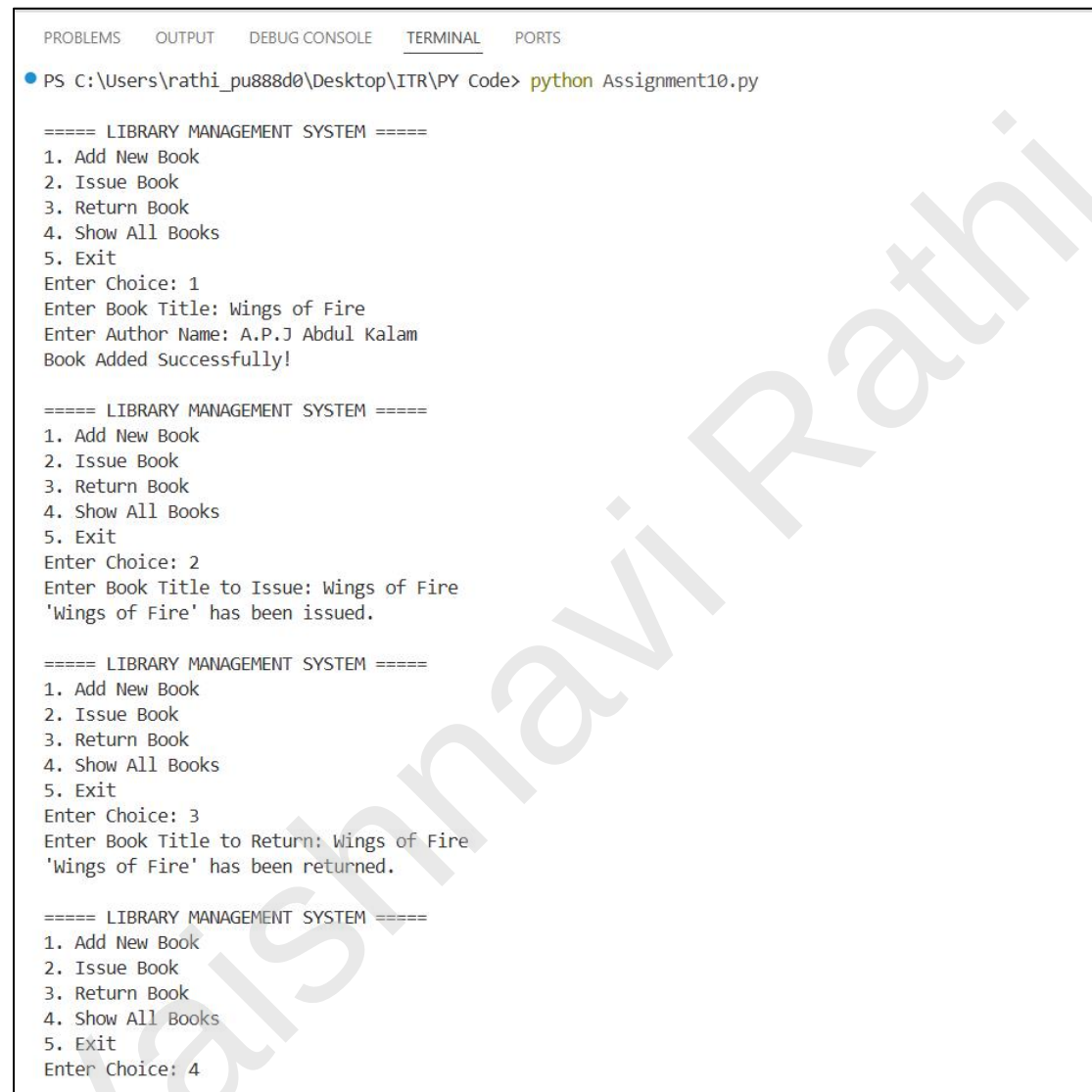
Fixed using f-string formatting.

Error 5:

name and age should be accessed using self.name and self.age.

Q10. Mini Project - Combined OOP

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment10.py

===== LIBRARY MANAGEMENT SYSTEM =====
1. Add New Book
2. Issue Book
3. Return Book
4. Show All Books
5. Exit
Enter Choice: 1
Enter Book Title: Wings of Fire
Enter Author Name: A.P.J Abdul Kalam
Book Added Successfully!

===== LIBRARY MANAGEMENT SYSTEM =====
1. Add New Book
2. Issue Book
3. Return Book
4. Show All Books
5. Exit
Enter Choice: 2
Enter Book Title to Issue: Wings of Fire
'Wings of Fire' has been issued.

===== LIBRARY MANAGEMENT SYSTEM =====
1. Add New Book
2. Issue Book
3. Return Book
4. Show All Books
5. Exit
Enter Choice: 3
Enter Book Title to Return: Wings of Fire
'Wings of Fire' has been returned.

===== LIBRARY MANAGEMENT SYSTEM =====
1. Add New Book
2. Issue Book
3. Return Book
4. Show All Books
5. Exit
Enter Choice: 4
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment10.py

===== BOOK LIST =====
Title : Wings of Fire
Author : A.P.J Abdul Kalam
Status : Available
-----

===== LIBRARY MANAGEMENT SYSTEM =====
1. Add New Book
2. Issue Book
3. Return Book
4. Show All Books
5. Exit
Enter Choice: 5
Thank You!
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 10 """
```

```
'''
```

Q10. (Mini Project - Combined OOP)

Create a Simple Library Management System using OOP.

```
'''
```

```
# Create Book class
```

```
class Book:
```

```
# Constructor to initialize book details
```

```
def __init__(self, title, author):
```

```
    self.title = title
```

```
    self.author = author
```

```
    self.status = "Available"
```

```
# Method to issue a book
```

```
def issue_book(self):
```

```
    if self.status == "Available":
```

```
        self.status = "Issued"
```

```
        print(f'{self.title}' has been issued.)
```

```

else:
    print(f"{self.title}' is already issued.")

# Method to return a book
def return_book(self):
    if self.status == "Issued":
        self.status = "Available"
        print(f"{self.title}' has been returned.")
    else:
        print(f"{self.title}' is already available.")

# Method to display book information
def show_info(self):
    print(f"Title : {self.title}")
    print(f"Author : {self.author}")
    print(f"Status : {self.status}")
    print("-" * 30)

# List to store all book objects
library = []

while True:

    print("\n===== LIBRARY MANAGEMENT SYSTEM =====")
    print("1. Add New Book")
    print("2. Issue Book")
    print("3. Return Book")
    print("4. Show All Books")
    print("5. Exit")

    choice = int(input("Enter Choice: "))

    # Add Book
    if choice == 1:
        title = input("Enter Book Title: ")
        author = input("Enter Author Name: ")

        book = Book(title, author)
        library.append(book)

        print("Book Added Successfully!")

    # Issue Book
    elif choice == 2:
        title = input("Enter Book Title to Issue: ")

        found = False

        for book in library:

```

```

        if book.title.lower() == title.lower():
            book.issue_book()
            found = True
            break

    if not found:
        print("Book Not Found!")

# Return Book
elif choice == 3:
    title = input("Enter Book Title to Return: ")

    found = False

    for book in library:
        if book.title.lower() == title.lower():
            book.return_book()
            found = True
            break

    if not found:
        print("Book Not Found!")

# Show All Books
elif choice == 4:

    if not library:
        print("No Books Available.")
    else:
        print("\n===== BOOK LIST =====")
        for book in library:
            book.show_info()

# Exit
elif choice == 5:
    print("Thank You!")
    break

else:
    print("Invalid Choice!")

```

"Explanation in Comments"

```
# Book class is created using OOP concepts.  
# __init__() initializes title, author and status.  
# issue_book() changes status from Available to Issued.  
# return_book() changes status from Issued to Available.  
# show_info() displays book details.  
# A list stores multiple book objects.  
# while loop creates a menu-driven program.  
# User can add, issue, return and view books.
```
